

Pre-Grouped Navigation Data

Technical Design and Performance Analysis

Overview

The `calculate_positions()` function performs the computationally intensive task of determining satellite positions and clock corrections for every observation in a RINEX dataset. While the original implementation leveraged multiprocessing to distribute work across CPU cores, the structure of the data passed to worker processes limited the effectiveness of parallel execution.

This optimization focused on reducing redundant navigation-data searches and minimizing the amount of data each worker process needed to access. The goal was to improve runtime scalability while preserving the existing multiprocessing architecture.

Original Implementation

The original implementation partitioned the observation DataFrame and distributed those partitions to worker processes.

```
obs_partition = np.array_split(self.obs, chunks)

with multiprocessing.Pool(processes=num_procs) as pool:
    results = list(
        pool.imap(
            self.get_satellite_position_process_worker,
            obs_partition
        )
    )
```

Each worker processed observations independently, but navigation lookups continued to operate against the full navigation DataFrame.

As a result, every observation calculation repeatedly searched a large shared dataset regardless of which satellites were actually present in the worker's partition.

Identified Performance Problems

1. Repeated Navigation Dataset Searches

The navigation DataFrame contains ephemeris records for all satellites contained within the dataset.

During position calculations, helper functions repeatedly searched this DataFrame to locate records corresponding to a specific satellite PRN.

This meant:

- The same navigation records were located repeatedly.
- Every worker searched irrelevant satellite records.
- Navigation lookup costs were paid many times throughout execution.

The resulting pattern was highly redundant:

```
Observation -> Search Navigation Data  
Observation -> Search Navigation Data  
Observation -> Search Navigation Data
```

Even though many observations referenced the same satellites.

2. Inefficient Worker Data Access

Each worker process only needed navigation information for the satellites contained within its assigned observations.

However, navigation lookups continued to operate against the complete navigation dataset.

This increased:

- Memory pressure
- Lookup overhead
- Multiprocessing serialization costs

while providing no computational benefit.

Optimization Strategy

Optimization 1: Pre-Group Navigation Records by PRN

Navigation data is grouped once before multiprocessing begins.

```
nav_by_prn = dict(tuple(self.nav.groupby(level='sv')))
```

This creates a dictionary mapping satellite PRNs to their corresponding navigation records.

Conceptually:

```
G01 -> Navigation Records
G02 -> Navigation Records
G03 -> Navigation Records
```

rather than repeatedly searching a single large DataFrame.

Benefits

- Eliminates repeated full-dataset scans.
- Converts repeated search operations into dictionary lookups.
- Moves work from the processing loop into a one-time setup step.
- Reduces redundant computation across worker processes.

Optimization 2: Worker-Specific Navigation Slicing

After grouping navigation records, each observation partition is paired only with navigation data relevant to satellites present within that partition.

```
partitions_with_nav = [
    (chunk, {
        prn: nav_by_prn[prn]
        for prn in chunk.index.get_level_values('sv').unique()
        if prn in nav_by_prn
    })
    for chunk in obs_partition
]
```

Workers therefore receive:

```
Observation Chunk
+
Relevant Navigation Records
```

instead of:

```
Observation Chunk
+
Entire Navigation Dataset
```

Benefits

- Reduces lookup scope.
 - Improves cache locality.
 - Reduces memory consumption.
 - Minimizes unnecessary data movement between processes.
-

Design Considerations

This optimization intentionally preserved the existing multiprocessing structure rather than introducing a new processing model.

The goal was to improve performance through better data organization and distribution while keeping the implementation understandable for future contributors.

Detailed comments were added to explain the reasoning behind navigation grouping and worker-specific data assignment so that future modifications can be made safely and with full awareness of the performance tradeoffs involved.